Doc. no.:	P1957R1
Date:	2020-1-10
Audience:	LWG, CWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

Converting from `T*` to `bool` should be considered narrowing (re: US 212)

Changes since R0

- propose a DR against C++11 rather than a breaking change in C++20.
- collect and analyze the impact of the change in Data and impact.

Background

LWG 3228 (https://cplusplus.github.io/LWG/issue3228) "Surprising `variant` construction" shows that, after applying P0608R3[1], a user-defined type that is convertible to `bool` now may construct an alternative type other than `bool`:

```
bitset<4> b("0101");
variant<bool, int> v = b[1];  // holds int
```

This is a direct result of a workaround introduced in R3 of the paper. The original issue found in R1 was that treating boolean conversion as narrowing conversion by detecting it is not implementable in the library. And the workaround was to ban conversions to `bool` naively.

Proposal

I propose to drop the workaround and treat a conversion from a pointer type or a pointer-to-member type to `bool` as narrowing conversion in the core language. The core language change should be a DR against C++11.

Discussion

If MSVC standard library implements P0608 today (libc++ and libstdc++ have shipped P0608R3) without special treatment of conversions to `bool` (equivalent to applying the proposed wording Part 2), we will end up with an ideal situation.

- `char*` argument cannot construct `bool` alternative,
- a user-defined type that is convertible to `char*` cannot construct `bool` alternative, and
- a user-defined type that is convertible to `bool` can construct a `bool` alternative.

Because MSVC considers non-literal pointer-to- `bool` conversion as a narrowing conversion:

```
// error C2397: conversion from 'char *' to 'bool' requires a narrowing conversion
bool y {new char()};
```

The rationale has been fully stated in N2215[2] when narrowing conversion was introduced to the standard in 2007:

> Some implicit casts, […] Others, such as **double->char** and **int*->bool**, are widely considered embarrassments.

> Our basic definition of narrowing (expressed using **decltype**) is that a conversion from **T** to **T2** is narrowing unless we can guarantee that
>
> ```
> T a = x;
> T2 b = a;
> T c = b;
> ```
>
> implies that **a==c** for every initial value **x**; that is, that the **T** to **T2** to **T** conversions are not value preserving.

One might argue that substituting in `T = char*` and `T2 = bool` will cause the 3rd line to fail to compile, which may suggest pointer-to- `bool` conversion to be slightly safer. Here is my response:

First, whether `T2` to `T` conversion is implicit or explicit does not change the fact that `T` to `T2` is not value preserving.

Second, when expressing what is not narrowing,

> […] or any combination of these conversions, except in cases where the initializer is a constant expression and the actual value will fit into the target object; that is, where **decltype(x)(T{x})==decltype(x)(x)**.

The wording also used casts.

Constant evaluation

There is a question of whether narrowing conversion should exclude some cases when pointers are converting to `bool` in a constant expression. According to N2215's rationale, only `(T*)nullptr` to `bool` is value preserving, but this is not what MSVC implemented. Rather than evaluating constant expressions, MSVC deems only literals to be non-narrowing.

I think neither tweak is necessary. The cases we may allow are at best as meaningful as

```
int x = {1.0};
```

About `nullptr`

"Conversion" from `std::nullptr_t` to `bool` is not involved in the discussion on narrowing conversions. It is not a boolean conversion, because `nullptr` is a *null pointer constant*, but not a *null pointer value* or *null member pointer value*. In short, `std::nullptr_t` is not convertible to `bool`, but `bool` is constructible from `std::nullptr_t`:

```
// error: converting to 'bool' from 'std::nullptr_t' requires direct-initializatio
bool x = nullptr;
bool y = {nullptr};    // ditto
bool z{nullptr};       // ok
```

GCC implemented these correctly.

Data and impact

In Belfast, Core expressed concern about this change if applied as a DR against C++11. The breakage is suspected as bad as adding narrowing conversion to C++11. The author coordinated with people in Google and Facebook to experiment with the change (LLVM#D64034 (https://reviews.llvm.org/D64034)) on their codebase as well as FreeBSD ports (source-based package management system). The results are collected below.

Google's codebase

Google's codebase found two breakages. One of which is in Chromium code referring to an old version of V8 and is not a bug. It looks like the following:

```
ParameterList result{{}, {}, context->VARARGS(), {}};
```

`ParameterList` is declared as

```
struct ParameterList {
  std::vector<std::string> names;
  std::vector<TypeExpression*> types;
  bool has_varargs;
  std::string arguments_variable;
};
```

and `VARARGS` is a member function that returns `antlr4::tree::TerminalNode*`. This breakage is also found in FreeBSD port `www/node10` (`www/node` is on version 13), where my proposed fix is:

```
ParameterList result{{}, {}, context->VARARGS() != nullptr, {}};
```

The other breakage is suspected to be a bug.

Facebook's codebase

Facebook's main codebase (excluding mobile, Oculus, etc.) failed 900 targets after applying the Clang patch. Semi-random samples point to the same issue, which is a bug:

```
template <typename T, size_t N>
auto count_unique(const std::array<T, N>& v)
{
    return std::unordered_set<T>{v.begin(), v.end()}.size();
}
```

When `T` is `bool` and `v.begin()` and `v.end()` are pointers (libc++ & libstdc++), the code above slightly constructs a set from two `bool` values without the patch.

Facebook is fixing their codebase and is happy to land the core language change.

FreeBSD ports collection

We found (https://bugs.freebsd.org/bugzilla/show_bug.cgi?id=242810) 7 ports failed, and 300 ports skipped due to these failures. Among those, one duplicates what appears in Google's codebase as aforementioned, two of them are the same because `www/webkit2-gtk3` has an issue while `java/openjfx8-devel` inlined a copy of WebKit.

The five distinct issues are the following:

1. avarice-2.13

```
// DEV_ATMEGA32M1
{
    "atmega32m1",
    0x9584,
    128, 256,       // 32768 bytes flash
    4, 256, // 1024 bytes EEPROM
    31 * 4, // 31 interrupt vectors
    DEVFL_MKII_ONLY,
    NULL,   // registers not yet defined
    atmega32m1_io_registers,
    0x00, 0x0000, // fuses
    {
        0   // no mkI support
    },
```

The struct definition starts with

```
typedef struct {
 const char* name;
 const unsigned int device_id;

 unsigned int flash_page_size;
 unsigned int flash_page_count;
 unsigned char eeprom_page_size;
 unsigned int eeprom_page_count;
 unsigned int vectors_end;
 enum dev_flags device_flags;

 gdb_io_reg_def_type *io_reg_defs;

 bool is_xmega;
```

The initializer to `io_reg_defs` should be either `NULL` or the `atmega32m1_io_registers` array of type `gdb_io_reg_def_type []`. The `atmega32m1_io_registers` array probably shouldn't initialize the `is_xmega` field. This is suspected a bug.

2. ecwolf-1.3.3

```
static const struct ExpressionFunction
{
    const char* const                               name;
    int                                             ret;
    unsigned short                                  args;
    bool                                            takesRNG;
    FunctionSymbol::ExprFunction    function;
} functions[] =
{
    { "cos",        TypeHierarchy::FLOAT,   1,      false,  ExprCos },
    { "frandom",    TypeHierarchy::FLOAT,   2,      true,   ExprFRandom },
    { "random",     TypeHierarchy::INT,     2,      true,   ExprRandom },
    { "sin",        TypeHierarchy::FLOAT,   1,      false,  ExprSin },
    { NULL, 0, false, NULL }
};
```

The last initializer clause misses an initializer such as `TypeHierarchy::VOID` between `NULL` and `0`. Note that `NULL` expands to `nullptr` on FreeBSD in the C++ code. However, Clang could have caught this issue if it correctly implements `nullptr`.

3. openorienteering-mapper-0.9.1

The code is intentional (using `{ }` as a cast) and is not a bug.

```
inline
constexpr ObjectPathCoord::operator bool() const
{
    return bool { object };
}
```

The following fix arguably conveys more information locally since the readers don't have to guess what `object` is.

```
    return object != nullptr;
```

4. ja-scim-anthy-1.2.7

   The code uses the wrong literals but is not a bug.

```
{
    SCIM_ANTHY_CONFIG_SHOW_TRAY_ICON,
    SCIM_ANTHY_CONFIG_SHOW_TRAY_ICON_DEFAULT,
    SCIM_ANTHY_CONFIG_SHOW_TRAY_ICON_DEFAULT,
    N_("Show _tray icon"),
    NULL,
    NULL,
    NULL,
},
{
    NULL,
    "",
    "",
    NULL,
    NULL,
    NULL,
    NULL,
    false,
},
```

   The struct to initialize is

```
struct BoolConfigData
{
    const char *key;
    bool        value;
    bool        default_value;
    const char *label;
    const char *title;
    const char *tooltip;
    void       *widget;
    bool        changed;
};
```

   Note that the file that triggers new errors has been patched already due to missed initializers.

5. webkit2-gtk3-2.26.2

```
if (view)
    m_mediaQueryEvaluator = MediaQueryEvaluator { view->mediaType() };
else
    m_mediaQueryEvaluator = MediaQueryEvaluator { "all" };
```

The expression `MediaQueryEvaluator { "all" }` triggers an error because overload resolution brings it to the following constructor.

```
class MediaQueryEvaluator {
public:
    // Creates evaluator which evaluates only simple media queries.
    // Evaluator returns true for "all", and returns value of
    // \mediaFeatureResult for any media features.
    explicit MediaQueryEvaluator(bool mediaFeatureResult = false);
```

And it is surprisingly intentional. It is not a (runtime) bug, but it also means that `MediaQueryEvaluator { "none" }` and `MediaQueryEvaluator { "all" }` have the same effect. This trick is refactored away in Blink.

After patching the seven ports, the previously skipped 300 ports build cleanly.

That's all the breakages we found so far.

Wording

The wording is relative to N4842.

Part 1

Modify 9.4.4 [dcl.init.list]/7 as follows:

A narrowing conversion is an implicit conversion

— from a floating-point type to an integer type, or

[...]

— from an integer type or unscoped enumeration type to an integer type that cannot represent all the values of the original type, except where the source is a constant expression whose value after integral promotions will fit into the target type~~.~~, or

— from a pointer type or a pointer-to-member type to `bool`.

[Note: ... — end note] [Example:

```
 ...
int ii = {2.0}; // error: narrows
float f1 { x }; // error: might narrow
float f2 { 7 }; // OK: 7 can be exactly represented as a float
bool b = {"meow"}; // error: narrows
int f(int);
int a[] = { 2, f(2), f(2.0) }; // OK: the double-to-int conversion is not at
```

— end example]

Modify C.2.3 [diff.cpp03.dcl.dcl]/2 as follows:

Effect on original feature: Valid C++ 2003 code may fail to compile in this International Standard. For example ~~, the following code is valid in C++ 2003 but invalid in this International Standard because~~ `double` ~~to~~ `int` ~~is a narrowing conversion~~:

```
int x[] = { 2.0 };    // ill-formed; previously well-formed
bool y[] = { "bc" };  // ill-formed; previously well-formed
```

Because `double` to `int` and `const char*` to `bool` are narrowing conversions.

Part 2

Modify 20.7.3.1 [variant.ctor]/12 as indicated:

```
template<class T> constexpr variant(T&& t) noexcept(see below );
```

> Let `T`$_j$ be a type that is determined as follows: build an imaginary function *`FUN`*`(T`$_i$`)` for each alternative type `T`$_i$ for which `T`$_i$ `x[] = {std::forward<T>(t)};` is well-formed for some invented variable `x` ~~and, if `T`$_i$ is *cv* `bool`, `remove_cvref_t<T>` is `bool`~~. […]

Modify 20.7.3.3 [variant.assign]/10 as indicated:

```
template<class T> variant& operator=(T&& t) noexcept(see below );
```

> Let `T`$_j$ be a type that is determined as follows: build an imaginary function *`FUN`*`(T`$_i$`)` for each alternative type `T`$_i$ for which `T`$_i$ `x[] = {std::forward<T>(t)};` is well-formed for some invented variable `x` ~~and, if `T`$_i$ is *cv* `bool`, `remove_cvref_t<T>` is `bool`~~. […]

Acknowledgments

Thank Eric Fiselier and Victor Zverovich for their hard work and thoughts.

References

1. Yuan, Zhihao. P0608R3 *A sane variant converting constructor*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0608r3.html (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0608r3.html) ↩

2. Stroustrup, Bjarne and Gabriel Dos Reis. N2215 *Initializer lists (Rev. 3)*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2215.pdf (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2215.pdf) ↩